LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

ABSTRACTION

Disusun Oleh:

Muhammad Fharel

2511531010

Kelas Praktikum B

Tanggal Praktikum: 2 November 2026

Dosen Pengampu:

Afdhal Dinilhak M.Kom

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

A. Identitas

Nama: Muhammad Fharel

NIM: 2511531010

Kelas: B

Output *tools.NimParams:*

```
PS E:\LAPRAK SEMESTER 3\LAPRAK PBO\praktikum-oop\p6
Parameter tugas untuk NIM 2511531010
  CARD_ADMIN_PERCENT  = 1
  TRANSFER_ADMIN_FEE  = 2500
  SMS_MAX_LENGTH      = 110
```

B. Penjelasan Kode Program

1) Latihan

1. L1 - Abstract class dan abstract method

   Pada latihan L1, dibuat sebuah sistem toko online UMKM sederhana yang menerapkan konsep abstraction. Sistem dirancang untuk mengelola berbagai jenis produk yang dijual serta menghitung biaya pengiriman berdasarkan berat barang.

   Sistem menggunakan abstract class Product untuk detail setiap product nantinya (sebagai kelas induk) dan class abstract class ShippingService untuk implementasi biaya pengirmanan di sistem.

No.	Nama Class	Jenis	Fungsi
1	*Product*	*Abstract class*	Kelas induk untuk seluruh produk.
2	*PhysicalProduct*	*Class*	Mengelola untuk *physical product*
3	*DigitalProduct*	*Class*	Mengelola untuk *digital product*
4	*ShippingService*	*Abstract class*	Kelas induk
5	*RegularShipping*	*Class*	

6	*ExpressShipping*	*Class*	Mengatur tarif pengiriman ekspres.

1. Product (src/shop/model/Product.java)

   Class product merupakan abstract class yang menyediakan struktur objek produk secara umum.

```java
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; // in rupiah

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name; }
    public long getPrice() { return price; }

    /** Price the customer actually pays. Subclasses may override it. */
    public long getFinalPrice() { return price; }

    /** Every concrete product must state its category. */
    public abstract String category();
```

```
32.
33.     /** A concrete method that relies on the abstract
   method category(). */
34.     public String getInfo() {
35.         return "[" + category() + "] " + name + " Rp" +
   getFinalPrice();
36.     }
37.}
```

Penjelasan class product:

1) public abstract class Product: Mencegah pemodelan objek instan langsung dari tipe umum Product.
2) Field sku, name, price: Diset secara private final guna menjamin enkapsulasi dan immutability field.
3) Protected Constructor Product: Konstruktor protected untuk dipanggil oleh subclass melalui klausa super(). Melakukan pengecekan validasi data.
4) Abstract Method category(): Abstract method tanpa badan kode yang mewajibkan seluruh kelas anak mendefinisikan label kategorinya.
5) Method getInfo(): Method konkret yang menyusun teks deskripsi produk dengan memanfaatkan hasil kembalian category().

2. PhysicalProduct (src/shop/model/PhysicalProduct.java)
   Merepresentasikan produk berbentuk fisik atau yang dapat dikirimkan kepada pelanggan dan menyimpan bobot massa weightGrams.

```
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int
weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) { throw new
IllegalArgumentException("Weight must be positive"); }
```

```
        this.weightGrams = weightGrams;
    }

    @Override
    public String category() { return "Fisik"; }

    @Override
    public int getWeightGrams() { return weightGrams; }
}
```

Penjelasan class PhysicalProduct:

1) extends Product: Menunjukkan bahwa PhysicalProduct mewarisi seluruh atribut dan method dari kelas induk Product.
2) Atribut berat produk: Disimpan dalam atribut weightGrams bertipe integer bertanda private final.
3) Konstruktor PhysicalProduct(): Menerima parameter SKU, nama, harga, dan bobot barang.
4) super(sku, name, price): Digunakan untuk memanggil konstruktor kelas induk Product guna menginisialisasi SKU, nama, dan harga.
5) Validasi weightGrams <= 0: Memastikan bobot bernilai bernilai positif (lebih besar dari nol).
6) @Override category(): Mengimplementasikan abstract method category() dengan mengembalikan string "Fisik".

3. DigitalProduct (src/shop/model/DigitalProduct.java)

   Kelas DigitalProduct merupakan turunan Product yang merepresentasikan produk berkas digital seperti e-book.

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;
```

```
    public DigitalProduct(String sku, String name, long price, String
downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) { throw new
IllegalArgumentException("URL must not be blank"); }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() { return downloadUrl; }

    @Override
    public String category() { return "Digital"; }

    @Override
    public int discountPercent() { return DIGITAL_DISCOUNT_PERCENT; }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

Penjelasan class DigitalProduct:

1) extends Product: Menandakan pewarisan dari kelas dasar Product.
2) Atribut downloadUrl: Atribut private final bertipe String untuk menyimpan alamat link unduhan.
3) Konstruktor DigitalProduct(): Menginisialisasi data digital produk dan memanggil super(...) kelas induk.
4) Validasi downloadUrl: Memastikan URL unduhan tidak kosong atau bernilai null.
5) Method getDownloadUrl(): Method getter untuk memperoleh tautan unduhan.
6) @Override category(): Mengimplementasikan abstract method category() dengan mengembalikan nilai "Digital".

4. ShippingService (src/shop/shipping/ShippingService.java)

Kelas ShippingService merupakan abstract class yang mendasari berbagai jenis layanan ekspedisi.

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;

    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() { return name; }

    /** Rate in rupiah per started kilogram. */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2 kg. */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) { return 0; }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

Penjelasan class ShippingService:

1) public abstract class ShippingService: Mendeklarasikan kelas abstrak layanan pengiriman yang tidak dapat diinstansiasi langsung.
2) Atribut name: Atribut private final untuk menyimpan nama jenis layanan ekspedisi.
3) Konstruktor ShippingService(): Menginisialisasi nama layanan serta memvalidasi agar tidak bernilai kosong.
4) Method getName(): Getter untuk mengambil nama layanan pengiriman.
5) Method ratePerKg(): Abstract method protected untuk menentukan tarif per kilogram yang wajib diisi oleh subclass.

6) Method calculateCost(): Method konkret untuk menghitung total biaya ekspedisi berbasis berat gram yang dibulatkan ke atas per kilogram dengan rumus (weightGrams + 999L) / 1000.

5. RegularShipping (src/shop/shipping/RegularShipping.java)
   Kelas RegularShipping merupakan turunan ShippingService yang mengimplementasikan opsi ekspedisi reguler.

```
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super("Reguler");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

Penjelasan class RegularShipping:

1) extends ShippingService: Menunjukkan bahwa RegularShipping mewarisi kelas induk ShippingService.
2) Konstruktor RegularShipping(): Memanggil super("Reguler") untuk menetapkan nama layanan ekspedisi sebagai pengiriman "Reguler".
3) @Override ratePerKg(): Mengimplementasikan abstract method induk dengan mengembalikan tarif sebesar Rp10.000 per kilogram.
4) Anotasi @Override: Menandakan bahwa method ini merupakan implementasi sah dari abstract method kelas induk.

6. ExpressShipping (src/shop/shipping/ExpressShipping.java)
   Kelas ExpressShipping merupakan turunan ShippingService yang mengimplementasikan opsi pengiriman cepat/ekspres.

```
package shop.shipping;
```

```java
public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super("Ekspres");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }
}
```

Penjelasan class ExpressShipping:

1) extends ShippingService: Menyatakan bahwa ExpressShipping merupakan subclass turunan dari ShippingService.
2) Konstruktor ExpressShipping(): Memanggil super("Ekspres") untuk memberikan identitas nama pengiriman ekspres.
3) @Override ratePerKg(): Mengimplementasikan abstract method dengan mengembalikan tarif sebesar Rp18.000 per kilogram.
4) Penggunaan calculateCost(): ExpressShipping otomatis menggunakan rumus kalkulasi pembulatan massa milik kelas induk ShippingService tanpa perlu menuliskan ulang kodenya.

2. L2 - Interface dan program to an interface

   Pada modul Latihan L2, arsitektur sistem toko online UMKM dikembangkan lebih lanjut dengan menerapkan antarmuka (*interface*). Konsep *interface* digunakan untuk mengikat kontrak perilaku (*behavioral contract*) tanpa bergantung pada struktur pewarisan kelas induknya. Di sini diperkenalkan antarmuka Shippable untuk menandai barang yang memiliki massa dan PaymentMethod sebagai abstraksi seluruh transaksi pembayaran. Selain itu, dibentuk kelas CheckoutService yang menerapkan prinsip *Program to an Interface* untuk mengelola kalkulasi transaksi belanja secara fleksibel.

No.	Nama Class	Jenis	Fungsi

1	*Shippable*	*Interface*	*Interface* untuk produk yang dapat dikirim.
2	*PhysicalProduct*	*Class*	Diubah agar mengimplementasikan interface *Shippable*.
3	*PaymentMethod*	*Interface*	*Interface* yang menjadi kontrak metode pembayaran.
4	*EWalletPayment*	*Class*	Pembayaran menggunakan *e-wallet*
5	*BankTransferPayment*	*Class*	Pembayaran melalui transfer bank.
6	*CheckoutService*	*Class*	Class untuk menghitung total belanja dan memproses pembayaran.
7	*ShopApp*	*Class*	Class utama untuk menjalankan dan menguji sistem toko.

1. Shippable (src/shop/shipping/Shippable.java)

   Shippable merupakan antarmuka (interface) yang dibuat untuk mengisolasi perilaku barang yang dapat dihitung massanya. Interface ini memungkinkan sistem pengiriman membaca berat suatu objek tanpa perlu mengetahui hierarki kelas produknya secara spesifik.

```
package shop.shipping;

public interface Shippable {
    int getWeightGrams();
}
```

Penjelasan class Shippable:

1) public interface Shippable: Mendeklarasikan antarmuka bernama Shippable.
2) int getWeightGrams(): Deklarasi abstract method tanpa bodi yang wajib diimplementasikan oleh kelas pengemban antarmuka untuk mengembalikan berat barang dalam satuan gram.

2. PhysicalProduct (src/shop/model/PhysicalProduct.java)
   Merepresentasikan produk berbentuk fisik atau yang dapat dikirimkan kepada pelanggan dan menyimpan bobot massa weightGrams.

```
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int
weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) { throw new
IllegalArgumentException("Weight must be positive"); }
        this.weightGrams = weightGrams;
    }

    @Override
    public String category() { return "Fisik"; }

    @Override
    public int getWeightGrams() { return weightGrams; }
}
```

Penjelasan class PhysicalProduct:

1) implements Shippable: Penanda bahwa PhysicalProduct tunduk pada kontrak antarmuka Shippable.

2) @Override public int getWeightGrams(): Realisasi implementasi method dari interface Shippable yang mengembalikan nilai dari variabel weightGrams.
3) @Override public String category(): Mengimplementasikan abstract method milik kelas induk Product dengan mengembalikan string label "Fisik".

3. PaymentMethod (src/shop/payment/PaymentMethod.java)

PaymentMethod bertindak sebagai antarmuka abstraksi untuk seluruh opsi saluran pembayaran. Seluruh metode pembayaran wajib memiliki nama identitas dan kemampuan memproses transaksi.

```
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);

    default long adminFee(long amount) {
        return 0;
    }
}
```

Penjelasan class PaymentMethod:

1) public interface PaymentMethod: Mendeklarasikan antarmuka untuk saluran pembayaran.
2) String getName(): Deklarasi method untuk memperoleh deskripsi atau nama jenis metode pembayaran.
3) boolean pay(long amount): Deklarasi method eksekusi pembayaran senilai amount, yang mengembalikan nilai kebenaran (true jika berhasil, false jika gagal).

4. EWalletPayment (src/shop/payment/EWalletPayment.java)

Kelas EWalletPayment mengimplementasikan PaymentMethod untuk mensimulasikan pemotongan saldo dompet digital konsumen.

```
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be
blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be
negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
```

```
        balance += amount;
        return true;
    }
}
```

Penjelasan class EWalletPayment:

1) implements PaymentMethod: Kelas ini memenuhi spesifikasi kontrak antarmuka pembayaran.
2) Atribut provider & balance: provider bersifat private final (penyedia layanan tidak dapat diubah), sedangkan balance bersifat mutable (saldo bertipe long dapat berkurang saat transaksi).
3) Konstruktor EWalletPayment(...): Menyimpan nama penyedia dan sisa saldo awal disertai pengecekan validasi.
4) @Override getName(): Mengembalikan format nama saluran e-wallet, seperti "E-Wallet Dompetku".
5) @Override pay(long amount): Logika eksekusi bayar. Transaksi ditolak (false) apabila nominal amount <= 0 atau tagihan melebihi sisa saldo balance. Jika valid, saldo dikurangi sebesar amount dan mengembalikan status true.

5. BankTransferPayment
   (src/shop/payment/BankTransferPayment.java)
   Kelas BankTransferPayment mengimplementasikan antarmuka PaymentMethod untuk mensimulasikan mekanisme transfer antarbank.

```
package shop.payment;

import shop.StudentInfo;

public class BankTransferPayment implements PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
        if (bankName == null || bankName.isBlank()) {
```

```
            throw new IllegalArgumentException("Bank name must not be
blank");
        }
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }

    // T1.c: Biaya admin tetap
    @Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }
}
```

Penjelasan class BankTransferPayment:

1) implements PaymentMethod: Menandakan pemenuhan kontrak antarmuka pembayaran.
2) Atribut bankName: Atribut private final penyimpan nama institusi bank (misal: Bank Nagari).
3) @Override getName(): Mengembalikan deskripsi metode pembayaran dalam format "Transfer " + bankName.
4) @Override pay(long amount): Menentukan bahwa transaksi transfer dianggap berhasil (true) selama nilai tagihan amount > 0.

6. CheckoutService (src/shop/service/CheckoutService.java)

CheckoutService mengelola logika transaksi belanja dengan menerapkan prinsip Program to an Interface. Kelas ini bekerja menggunakan acuan interface (Shippable, PaymentMetho.

d) sehingga tidak terikat ketat pada kelas konkret tertentu.

```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public long grandTotal(List<Product> items, ShippingService shipping,
PaymentMethod payment) {
        long baseTotal = total(items, shipping);
        return baseTotal + payment.adminFee(baseTotal);
    }

    public boolean checkout(List<Product> items, ShippingService
shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
```

```
        long amountToPay = grandTotal(items, shipping, payment);
        return payment.pay(amountToPay);
    }

    public boolean refund(RefundablePayment payment, long amount) {
        return payment.refund(amount);
    }
}
```

Penjelasan class CheckoutService:

1) Method subtotal(List<Product> items): Melakukan perulangan pada seluruh item keranjang belanja untuk menghitung total kumulatif harga akhir produk (getFinalPrice())
2) Method totalWeight(List<Product> items): Memeriksa item yang mengimplementasikan interface Shippable menggunakan pembanding instanceof (pattern matching), lalu menjumlahkan akumulasi beratnya dalam satuan gram.
3) Method total(List<Product> items, ShippingService shipping): Menghitung grand total pembayaran gabungan antara subtotal barang dan ongkos kirim ekspedisi.
4) Method checkout(...): Menerima parameter berjenis interface PaymentMethod. Ini merupakan contoh Program to an Interface, di mana method ini dapat memproses berbagai saluran pembayaran tanpa perlu diubah saat ada metode pembayaran baru.

7. ShopApp (src/shop/app/ShopApp.java)

   ShopApp bertindak sebagai kelas utama pemanggil (*driver class*) untuk melakukan pengujian alur kerja transaksi pada modul L2.

   Pada ShopApp, program total Reguler masih 265000 dan e-book masih 50000, digunakan sebagai pengujian untuk menghitung subtotal, berat produk, dan total belanja berdasarkan layanan pengiriman.

3. L3 - Interface Lanjutan: Default, Static, Private Method, dan Interface Inheritance

   Pada modul L3, antarmuka dikembangkan lebih lanjut memanfaatkan fitur Java modern: default method, static method, private method, serta pewarisan antar-interface (interface inheritance). Fitur diskon, refund, dan cicilan kartu kredit ditambahkan ke dalam sistem.

No.	Nama Class	Jenis	Fungsi
1	*Discountable*	*Interface*	*Interface* untuk mengatur pemberian diskon produk.
2	*DigitalProduct*	*Class*	Dimodifikasi agar mengimplementasikan *Discountable*.
3	*RefundablePayment*	*Interface*	*Interface* turunan *PaymentMethod* *(*mendukung *refund).*
4	*InstallmentPayment*	*Class*	*Interface* turunan *PaymentMethod* *(cicilan),*
5	*EWalletPayment*	*Class*	Dimodifikasi agar mendukung fitur *refund*.
6	*CreditCardPayment*	*Class*	Class pembayaran kartu kredit dengan fitur *refund* dan cicilan.
7	*CheckoutService*	*Class*	Ditambahkan method untuk memproses refund

8	*ShopApp*	*Class*	Dilengkapi dengan pengujian pembayaran, cicilan, dan refund.

1. Discountable (src/shop/discount/Discountable.java)

   Interface Discountable dirancang untuk mengelola mekanisme kalkulasi diskon pada produk toko. Interface ini mengombinasikan abstract method untuk menentukan besaran persentase diskon, default method untuk memberikan implementasi langsung perhitungan diskon tanpa memaksa kelas pengisi mengimplementasikannya ulang, static method sebagai fungsi pembantu kalkulasi matematika, serta private method untuk memvalidasi batas aman persentase diskon secara internal.

```
package shop.discount;

public interface Discountable {
    int discountPercent(); // abstract: each class decides

    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }
}
```

Penjelasan class Discountable:

1) int discountPercent(): Abstract method tanpa bodi yang menuntut kelas pengemban antarmuka untuk menentukan nilai persentase diskon.
2) default long applyDiscount(long price): Default method yang menyediakan kalkulasi bawaan untuk menghitung nilai harga bersih setelah dipotong diskon.
3) static long percentOf(long amount, int percent): Static method pembantu (utility method) yang dapat dipanggil langsung untuk menghitung besaran nilai dari persentase diskon.
4) private int safePercent(): Private method internal antarmuka yang mengamankan persentase diskon agar selalu berada dalam batas valid (antara 0% hingga 100%).

2. DigitalProduct (src/shop/model/DigitalProduct.java)
   Kelas DigitalProduct pada modul L3 mengalami modifikasi dengan menambahkan klausa implements Discountable. Modifikasi ini bertujuan agar setiap produk digital (seperti e-book) secara otomatis mendapatkan fasilitas potongan harga sebesar 10% yang dihitung secara dinamis pada saat pemanggilan harga akhir produk.

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;

    public DigitalProduct(String sku, String name, long price, String
downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) { throw new
IllegalArgumentException("URL must not be blank"); }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() { return downloadUrl; }
```

```
    @Override
    public String category() { return "Digital"; }

    @Override
    public int discountPercent() { return DIGITAL_DISCOUNT_PERCENT; }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

Penjelasan class DigitalProduct:

1) implements Discountable: Menunjukkan bahwa DigitalProduct menerapkan antarmuka diskon.
2) DIGITAL_DISCOUNT_PERCENT = 10: Konstanta private static final yang menetapkan persentase diskon default produk digital sebesar 10%.
3) @Override discountPercent(): Mengimplementasikan method dari Discountable dengan mengembalikan nilai persentase diskon (10%).
4) @Override getFinalPrice(): Meng-override method milik kelas induk Product dengan memanggil applyDiscount(getPrice()). Dengan demikian, harga akhir e-book yang semula Rp50.000 otomatis menjadi Rp45.000 tanpa perlu mengubah logika perhitungan pada CheckoutService.

3. RefundablePayment (src/shop/payment/RefundablePayment.java)

   Interface RefundablePayment merupakan antarmuka turunan dari PaymentMethod (interface inheritance). Antarmuka ini dibentuk khusus untuk mendefinisikan spesifikasi saluran pembayaran yang memiliki kemampuan pengembalian dana (refund) serta menyediakan deskripsi kebijakan refund standar.

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);
```

```
    default String policy() {
        return "Refund maksimal 7 hari";
    }
}
```

Penjelasan class RefundablePayment:

1) extends PaymentMethod: Menunjukkan pewarisan antarmuka di mana RefundablePayment mewarisi seluruh method abstrak dari PaymentMethod (getName() dan pay()).
2) boolean refund(long amount): Method abstrak baru yang wajib diisi oleh saluran pembayaran yang mendukung refund.
3) default String policy(): Default method yang menyediakan teks deskripsi kebijakan bawaan pengembalian dana, yaitu "Refund maksimal 7 hari".

4. InstallmentPayment (src/shop/payment/InstallmentPayment.java)

nterface InstallmentPayment merupakan antarmuka turunan dari PaymentMethod yang digunakan untuk mendefinisikan kemampuan pembayaran secara tertangsur atau cicilan. Antarmuka ini menyediakan logika pembagian angsuran bulanan yang dibulatkan ke atas serta memvalidasi durasi tenor cicilan yang diizinkan.

```
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException("Months must be between 1
and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }

    default String policy() {
```

```
        return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
    }
}
```

Penjelasan class InstallmentPayment:

1) int maxInstallmentMonths(): Method abstrak untuk menentukan batas maksimal durasi cicilan dalam satuan bulan.
2) default long monthlyInstallment(long amount, int months): Default method untuk menghitung nominal angsuran per bulan. Method ini menyertakan validasi agar durasi bulan tidak kurang dari 1 atau melebihi batas maksimal, serta menggunakan rumus (amount + months - 1) / months untuk memastikan pembulatan ke atas pada nilai pecahan desimal.
3) default String policy(): Default method yang mengembalikan teks deskripsi kebijakan durasi cicilan.

5. EWalletPayment (src/shop/payment/EWalletPayment.java)

   Pada modul L3, deklarasi kelas EWalletPayment diperbarui dari implements PaymentMethod menjadi implements RefundablePayment. Perubahan ini dilakukan agar saluran pembayaran e-wallet mampu mengembalikan saldo pengguna ketika transaksi diproses untuk refund.

```
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be
blank");
        }
        if (balance < 0) {
```

```
            throw new IllegalArgumentException("Balance must not be
negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

Penjelasan class EWalletPayment:

1) implements RefundablePayment: Kelas ini sekarang terikat pada antarmuka pembayaran yang mendukung pengembalian dana.
2) @Override refund(long amount): Mengimplementasikan fungsi refund. Apabila nominal pengembalian amount > 0, proses pengembalian dana dinyatakan berhasil (true) dan saldo balance akan bertambah sebesar nilai amount.

6. CreditCardPayment (src/shop/payment/CreditCardPayment.java)

   Kelas CreditCardPayment dibuat dengan mengimplementasikan dua antarmuka turunan sekaligus, yaitu RefundablePayment dan InstallmentPayment. Kelas ini mengelola transaksi limit kredit pengguna dan mendemonstrasikan penyelesaian masalah diamond conflict yang timbul akibat bentrokan nama method policy() dari kedua antarmuka induknya.

```
package shop.payment;

import shop.StudentInfo;

public class CreditCardPayment implements RefundablePayment,
InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        if (cardHolder == null || cardHolder.isBlank()) {
            throw new IllegalArgumentException("Card holder must not be
blank");
        }
        if (creditLimit <= 0) {
            throw new IllegalArgumentException("Credit limit must be
positive");
        }
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    public long getUsedCredit() {
        return usedCredit;
    }

    @Override
```

```
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount > creditLimit) {
            return false;
        }
        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }

    @Override
    public long adminFee(long amount) {
        return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
    }

    @Override
    public String policy() {
        return RefundablePayment.super.policy() + "; " +
InstallmentPayment.super.policy();
    }
}
```

Penjelasan class CreditCardPayment:

1) implements RefundablePayment, InstallmentPayment: Mengimplementasikan dua antarmuka sekaligus sehingga mendukung fitur refund dan cicilan.
2) @Override pay(long amount): Memproses pemakaian kredit. Pembayaran berhasil jika amount > 0 dan total kredit yang dipakai (usedCredit + amount) tidak melampaui batas limit creditLimit.

3) @Override refund(long amount): Mengurangi besaran usedCredit ketika terjadi pengembalian dana yang valid.
4) @Override maxInstallmentMonths(): Menetapkan batas maksimal durasi cicilan kartu kredit selama 12 bulan.
5) @Override policy(): Di-override secara eksplisit untuk menyelesaikan bentrokan diamond conflict dengan menggabungkan keluaran kedua default method milik antarmuka induk menggunakan sintaks RefundablePayment.super.policy() dan InstallmentPayment.super.policy().

7. CheckoutService (src/shop/service/CheckoutService.java)

Pada kelas CheckoutService, ditambahkan method refund() untuk meneruskan eksekusi pemrosesan pengembalian dana ke objek metode pembayaran yang relevan. Method ini memanfaatkan keunggulan type safety pada Java untuk memastikan hanya jenis pembayaran bertipe RefundablePayment yang dapat memproses pengembalian uang.

```
public boolean refund(RefundablePayment payment, long amount) {
        return payment.refund(amount);
    }
```

Penjelasan class CheckoutService:

Method refund() menerima parameter payment bertipe RefundablePayment dan memanggil fungsi refund(amount) dari objek tersebut. Saluran pembayaran yang tidak mengimplementasikan RefundablePayment (seperti BankTransferPayment) akan ditolak secara otomatis oleh penyusun program (compiler) saat penulisan kode.

8. ShopApp (src/shop/app/ShopApp.java)

Pada kelas utama ShopApp, ditambahkan serangkaian skenario pengujian untuk memvalidasi seluruh fitur baru yang dibangun pada

L3. Pengujian meliputi pemeriksaan harga e-book yang terpotong diskon 10%, simulasi eksekusi transaksi pembayaran via e-wallet dan kartu kredit, kalkulasi cicilan bulanan, pemrosesan refund, serta pencetakan deskripsi kebijakan pembayaran.

4. L4 - final method dan final class

Pada latihan L4, sistem toko online UMKM dikembangkan dengan menerapkan kata kunci (keyword) final untuk membatasi pewarisan kelas dan modifikasi method. Penggunaan kata kunci final dibagi menjadi dua penerapan utama: final method pada ShippingService untuk mengunci logika kalkulasi ongkos kirim agar uniform dan tidak dapat diubah oleh subkelas mana pun, serta final class pada kelas Address untuk membentuk kelas yang bersifat mutlak immutable (tidak dapat diwarisi dan nilainya tidak dapat diubah setelah diinstansiasi). Selain itu, dibuat kelas uji coba FreeShipping untuk membuktikan mekanisme perlindungan final method terhadap kesalahan override.

No.	Nama Class	Jenis	Fungsi
1	*ShippingService*	*Interface*	*Interface* untuk produk yang dapat dikirim.
2	*ShippingService*	*Class*	Class baru dengan sifat final dan atribut immutable.
3	*FreeShipping*	*Class*	Class sementara untuk menguji error ketika mencoba override method final.

1. ShippingService (src/shop/shipping/ShippingService.java)

Pada modul L4, kelas abstrak ShippingService diperbarui pada bagian method calculateCost(). Penambahan modifikator final pada method tersebut bertujuan untuk menetapkan standar baku rumus perhitungan biaya pengiriman toko berbasis pembulatan massa kilogram ke atas. Dengan demikian, seluruh subkelas ekspedisi dipaksa untuk menginduk pada rumus kalkulasi dasar yang sama, tetapi tetap diberikan kebebasan untuk menentukan besaran tarif per kilogramnya via method ratePerKg().

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;

    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() { return name; }

    /** Rate in rupiah per started kilogram. */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2 kg. */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) { return 0; }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

Penjelasan class ShippingService:

1) public final long calculateCost(int weightGrams): Penggunaan kata kunci final pada method ini mengunci implementasi rumus kalkulasi ongkir. Subclass turunan seperti RegularShipping atau ExpressShipping dilarang untuk meng-overide logika hitung ini.

2) Validasi weightGrams <= 0: Memastikan jika massa barang fisik tidak valid (0 atau negatif), maka biaya pengiriman secara otomatis mengembalikan angka 0.
3) Logika Pembulatan (weightGrams + 999L) / 1000: Mengubah satuan massa dari gram ke kilogram utuh dengan pembulatan ke atas (contoh: 300 gram dianggap 1 kg, 1200 gram dianggap 2 kg).
4) Perhitungan kilograms * ratePerKg(): Mengalikan hasil pembulatan kilogram dengan tarif spesifik yang dikembalikan oleh subclass masing-masing.

2. FreeShipping (src/shop/shipping/FreeShipping.java)
   Class FreeShipping digunakan untuk menguji perlindungan final method terhadap percobaan modifikasi rumus pengiriman.

```
@Override
    public long calculateCost(int weightGrams) {
        return 0;
    }
```

Penjelasan class FreeShipping:
Ketika program dikompilasi, muncul error karena method calculateCost() sudah menggunakan keyword final. Artinya, subclass tidak diperbolehkan mengubah implementasi method tersebut.

3. Address (src/shop/model/Address.java)
   Class Address dibuat menggunakan final class untuk menyimpan data lokasi pengiriman pesanan secara aman dan konstan.

```
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {
```

```
        if (street == null || street.isBlank() || city == null ||
city.isBlank() || postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Street, city, and postal
code must not be blank");
        }
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() { return street; }
    public String getCity() { return city; }
    public String getPostalCode() { return postalCode; }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Penjelasan class Address:

1) public final class Address: Mencegah kelas Address diwarisi oleh kelas lain.
2) Atribut private final & Tanpa Setter: Menjamin sifat immutable mutlak agar data alamat tidak bisa diubah setelah objek dibuat.
3) @Override toString(): Mengembalikan format string alamat lengkap (jalan, kota kode_pos).

5. L5 - Nested Class: Static Nested, Inner Class, dan Anonymous Class

   Pada modul L5, program dikembangkan dengan menerapkan tiga jenis nested class, yaitu static nested class, inner class, dan anonymous class. Ketiganya digunakan untuk mengelompokkan kelas pembantu secara terstruktur sesuai dengan fungsi dan tingkat keterikatannya dengan kelas utama.

No.	Nama Class	Jenis	Fungsi

1	*Order*	*Class*	*Class* baru untuk menyimpan data pesanan.
2	*Order.Line*	*Class Order*	*Static nested class* untuk menyimpan detail barang pesanan.
3	*Order.Receipt*	*Class Order*	*Inner class* untuk membuat struk pesanan.
4	*ShopApp*	*Class*	Diubah untuk menggunakan *anonymous class* sebagai metode pembayaran.

1. Order (src/shop/model/Order.java)

   Class Order mengelola seluruh alur pesanan yang di dalamnya menampung *static nested class* Line dan *inner class* Receipt.

```
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank() || shippingAddress ==
null) {
            throw new IllegalArgumentException("Order id and address are
required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }
```

```
    public String getOrderId() { return orderId; }
    public Address getShippingAddress() { return shippingAddress; }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }

    public long getItemsTotal() {
        long total = 0;
        for (Line line : lines) {
            total += line.subtotal();
        }
        return total;
    }

    public Receipt createReceipt() {
        return new Receipt();
    }

    public static class Line {
        private final Product product;
        private final int quantity;

        public Line(Product product, int quantity) {
            if (product == null || quantity <= 0) {
                throw new IllegalArgumentException("Product must not be
null and quantity must be positive");
            }
            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() { return product; }
        public int getQuantity() { return quantity; }

        public long subtotal() {
            return product.getFinalPrice() * quantity;
        }
    }
```

```
    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("STRUK PESANAN ").append(orderId).append("\n");
            for (Line line : lines) {
                sb.append("  ").append(line.getProduct().getName())
                  .append(" x").append(line.getQuantity())
                  .append(" = Rp").append(line.subtotal()).append("\n");
            }
            sb.append("Total barang:
Rp").append(getItemsTotal()).append("\n");
            sb.append("Kirim ke: ").append(shippingAddress);
            return sb.toString();
        }
    }
}
```

Penjelasan class Order:

1) Class Order: Menyimpan ID transaksi (orderId), alamat pengiriman Address, dan daftar barang pesanan (lines).
2) Static Nested Class Line: Menampung data produk dan jumlah barang tanpa memerlukan referensi atau instance dari kelas luar Order.
3) Method getItemsTotal(): Menghitung akumulasi total subtotal dari seluruh barang di daftar pesanan.
4) Inner Class Receipt: Dideklarasikan tanpa kata kunci static sehingga dapat mengakses variabel orderId dan shippingAddress milik kelas luar Order secara langsung via Order.this.
5) Method createReceipt(): Method untuk membuat objek Receipt guna mencetak struk transaksi.

2. Anonymous Class pada ShopApp (src/shop/app/ShopApp.java)

   Penggunaan anonymous class diterapkan pada kelas utama ShopApp untuk membuat objek instan dari interface PaymentMethod secara langsung.

```
PaymentMethod voucher = new PaymentMethod() {
            private long remaining = 50_000;

            @Override
```

```
            public String getName() {
                return "Voucher Promo Hari Ini";
            }

            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) { return false; }
                remaining -= amount;
                return true;
            }
        };
```

Penjelasan Anonymous Class pada ShopApp:

1) new PaymentMethod() { ... }: Membuat anonymous class yang mengimplementasikan antarmuka PaymentMethod secara on-the-fly tanpa perlu membuat file .java baru.
2) @Override getName() & pay(...): Mengisi implementasi method dari PaymentMethod secara spesifik untuk kebutuhan transaksi menggunakan voucher sekali pakai.

2) Tugas

1. T1 - Evolusi API dan konflik diamond

   Modul T1 mensimulasikan evolusi antarmuka (API evolution) dengan menambahkan biaya administrasi pembayaran serta menangani masalah bentrokan method bawaan (diamond conflict).

   a. T1.a - Menambahkan Abstract Method adminFee

      Penambahan method abstrak long adminFee(long amount) ke dalam PaymentMethod menyebabkan 4 compiler error pada kelas pengisi yang belum memiliki implementasinya, yaitu BankTransferPayment, CreditCardPayment, EWalletPayment, dan anonymous class voucher pada ShopApp.

```
public interface PaymentMethod {
    String getName();
    boolean pay(long amount);
    long adminFee(long amount);
}
```

b. T1.b – Mengganti menjadi Default method

Untuk menghindari breaking change, adminFee() diubah menjadi default method dengan nilai imbalan 0 sehingga kelas pengisi tidak wajib menuliskan ulang kodenya.

```
default long adminFee(long amount) {
        return 0;
    }
```

c. T1.c – Override Biaya Admin Sesuai Metode Pembayaran

Setiap saluran pembayaran menentukan tarif biayanya masing-masing:

- CreditCardPayment: Dihitung secara persentase dari total transaksi.

```
@Override
    public long adminFee(long amount) {
        return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
    }
```

- BankTransferPayment: Menggunakan biaya nominal tetap.

```
@Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }
```

- EWalletPayment: Class EWalletPayment tidak perlu diubah karena menggunakan default method dari PaymentMethod, sehingga biaya adminnya adalah Rp0.

d. T1.d – Menambahkan grandTotal() pada CheckoutService

CheckoutService diperbarui dengan method grandTotal() untuk menjumlahkan total harga barang, ongkos kirim, dan biaya admin. Method checkout() juga disesuaikan agar mengeksekusi pembayaran senilai grandTotal().

```
public long grandTotal(List<Product> items, ShippingService shipping,
PaymentMethod payment) {
        long baseTotal = total(items, shipping);
        return baseTotal + payment.adminFee(baseTotal);
    }
```

```
public boolean checkout(List<Product> items, ShippingService shipping,
PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        long amountToPay = grandTotal(items, shipping, payment);
        return payment.pay(amountToPay);
    }
```

e. T1.e - Konflik Diamond pada Default Method

Penambahan default method policy() pada RefundablePayment dan InstallmentPayment memicu diamond conflict di kelas CreditCardPayment. Penyelesaiannya dilakukan dengan meng-override policy() secara eksplisit pada CreditCardPayment:

```
default String policy() {
        return "Refund maksimal 7 hari";
    }
```

```
default String policy() {
        return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
    }
```

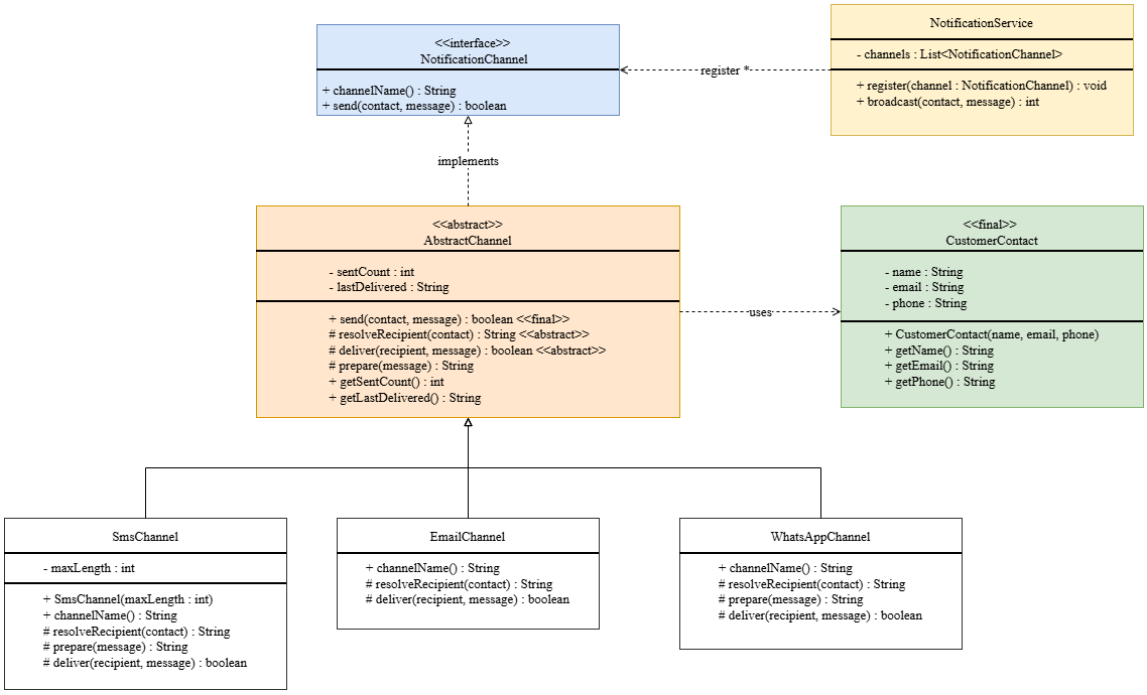
2. T2 - Merancang Sistem Notifikasi Pesanan

Modul T2 merancang sistem pengiriman notifikasi status pesanan multi-kanal (Email, SMS, WhatsApp) pada package shop.notification menggunakan kombinasi interface, abstract class (Template Method Pattern), dan final class.

No.	Nama Class	Jenis	Fungsi
1	*CustomerContact*	*Class*	Menyimpan data nama, *email*, dan nomor telepon pelanggan.
2	*NotificationChannel*	*Interface*	*Interface* sebagai kontrak untuk

			setiap kanal notifikasi.
3	*AbstractChannel*	*Abstract Class*	*Abstract class* yang menyediakan proses pengiriman notifikasi secara umum
4	*EmailChannel*	*Class*	Mengirim notifikasi melalui email.
5	*SmsChannel*	*Class*	Mengirim notifikasi melalui SMS dengan batas panjang pesan.
6	*WhatsAppChannel*	*Class*	Mengirim notifikasi melalui WhatsApp.
7	*NotificationService*	*Class*	Mengelola kanal dan mengirim notifikasi ke pelanggan.

Berikut merupakan rancangan UML yang menggambarkan hubungan antar class pada sistem notifikasi.



a. CustomerContact (src/shop/notification/CustomerContact.java)

Class immutable penyimpan profil kontak pelanggan.

```java
package shop.notification;

public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;

    public CustomerContact(String name, String email, String phone) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
        this.email = (email == null) ? "" : email;
        this.phone = (phone == null) ? "" : phone;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPhone() { return phone; }
}
```

Penjelasan class CustomerContact:

Atribut private final tanpa setter menjaga konsistensi data kontak agar tidak dapat dimutasi setelah dibuat.

b. NotificationChannel (src/shop/notification/NotificationChannel.java)

Interface kontrak dasar saluran notifikasi.

```java
package shop.notification;

public interface NotificationChannel {
    String channelName();
    boolean send(CustomerContact contact, String message);
}
```

Penjelasan class NotificationChannel:

Menentukan method channelName() dan send() yang wajib dimiliki seluruh tipe kanal.

c. AbstractChannel (src/shop/notification/AbstractChannel.java)

Abstract class pengatur alur kerja (Template Method).

```java
package shop.notification;

public abstract class AbstractChannel implements NotificationChannel {
    private int sentCount = 0;
    private String lastDelivered = "";

    protected abstract String resolveRecipient(CustomerContact contact);
    protected abstract boolean deliver(String recipient, String message);

    protected String prepare(String message) {
        return message;
    }

    @Override
    public final boolean send(CustomerContact contact, String message) {
        if (message == null || message.isBlank()) {
            return false;
        }
        String recipient = resolveRecipient(contact);
        if (recipient == null || recipient.isBlank()) {
            return false;
```

```
        }
        String preparedMessage = prepare(message);
        boolean success = deliver(recipient, preparedMessage);
        if (success) {
            sentCount++;
            lastDelivered = preparedMessage;
        }
        return success;
    }

    public int getSentCount() { return sentCount; }
    public String getLastDelivered() { return lastDelivered; }
}
```

Penjelasan class AbstractChannel:

Method send() dibuat final agar urutan validasi, pemrosesan pesan, dan pencatatan riwayat tidak dapat di-override oleh subkelas.

d. EmailChannel (src/shop/notification/EmailChannel.java)

Email hanya dapat dikirim jika alamat email pelanggan mengandung karakter @.

```
package shop.notification;

public class EmailChannel extends AbstractChannel {
    @Override
    public String channelName() { return "EMAIL"; }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        return (contact.getEmail().contains("@")) ? contact.getEmail() :
null;
    }

    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("[EMAIL to " + recipient + "] " + message);
        return true;
    }
}
```

Penjelasan class EmailChannel:

Memvalidasi alamat email harus mengandung karakter @ sebelum dikirim.

e. SmsChannel (src/shop/notification/SmsChannel.java)

```
package shop.notification;

public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel(int maxLength) {
        this.maxLength = maxLength;
    }

    @Override
    public String channelName() { return "SMS"; }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        return (contact.getPhone().startsWith("08")) ? contact.getPhone()
: null;
    }

    @Override
    protected String prepare(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength - 3) + "...";
        }
        return message;
    }

    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("[SMS to " + recipient + "] " + message);
        return true;
    }
}
```

Penjelasan class SmsChannel:

Memvalidasi nomor diawali "08" dan memotong panjang pesan jika melebihi batas maxLength.

f. WhatsAppChannel (src/shop/notification/WhatsAppChannel.java)

```
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {
    @Override
    public String channelName() { return "WA"; }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        if ((phone.startsWith("08") || phone.startsWith("+62")) &&
phone.length() >= 10) {
            return phone;
        }
        return null;
    }

    @Override
    protected String prepare(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("[WA to " + recipient + "] " + message);
        return true;
    }
}
```

Penjelasan class WhatsAppChannel:

Menerima nomor diawali "08"/"+62" minimal 10 digit serta menambahkan prefix "[Toko UMKM] ".

g. NotificationService (src/shop/notification/NotificationService.java)

```
package shop.notification;

import java.util.ArrayList;
import java.util.List;

public class NotificationService {
```

```
    private final List<NotificationChannel> channels = new ArrayList<>();

    public void register(NotificationChannel channel) {
        if (channel != null) {
            channels.add(channel);
        }
    }

    public int broadcast(CustomerContact contact, String message) {
        int count = 0;
        for (NotificationChannel channel : channels) {
            if (channel.send(contact, message)) {
                count++;
            }
        }
        return count;
    }
}
```

Penjelaan class NotificationService:

Mengelola daftar kanal berbasis antarmuka List<NotificationChannel> dan mengirimkan pesan ke seluruh kanal terdaftar.

h. TugasApp.java (src/shop/app/TugasApp.java)

Class TugasApp bertindak sebagai kelas penguji utama untuk menguji fungsionalitas pengiriman notifikasi pesanan (T2) sekaligus kalkulasi total biaya pembayaran dengan biaya administrasi (T1).

```
package shop.app;

import java.util.List;
import shop.StudentInfo;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.notification.*;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
```

```java
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.ShippingService;

public class TugasApp {
    public static void main(String[] args) {
        System.out.println("Tugas oleh " + StudentInfo.NAME + " (" +
StudentInfo.NIM + ")");

        Product coffee = new PhysicalProduct("KOPI-001", "Kopi Arabika
Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002", "Rendang Kemasan
250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-book Resep Kue
Nusantara", 50_000, "https://example.com");
        List<Product> cart = List.of(coffee, rendang, ebook);

        CheckoutService checkout = new CheckoutService();
        ShippingService express = new ExpressShipping();

        PaymentMethod wallet = new EWalletPayment("Dompetku", 300_000);
        PaymentMethod transfer = new BankTransferPayment("Bank Nagari");
        CreditCardPayment card = new CreditCardPayment("Siti Rahma",
1_000_000);

        System.out.println("\n== T1: Total dengan biaya admin (Ekspres)
==");
        System.out.println(wallet.getName() + " Rp" +
checkout.grandTotal(cart, express, wallet) + " (admin Rp" +
wallet.adminFee(checkout.total(cart, express)) + ")");
        System.out.println(transfer.getName() + " Rp" +
checkout.grandTotal(cart, express, transfer) + " (admin Rp" +
transfer.adminFee(checkout.total(cart, express)) + ")");
        System.out.println(card.getName() + ": Rp" +
checkout.grandTotal(cart, express, card) + " (admin Rp" +
card.adminFee(checkout.total(cart, express)) + ")");

        System.out.println("Kebijakan kartu kredit : " + card.policy());
        System.out.println("Kebijakan e-wallet      : " +
((EWalletPayment) wallet).policy());

        System.out.println("\n== T2: Notifikasi status pesanan ==");
        NotificationService service = new NotificationService();
        service.register(new EmailChannel());
        service.register(new SmsChannel(StudentInfo.SMS_MAX_LENGTH));
```

```
        service.register(new WhatsAppChannel());

        // Anonymous class untuk Log Konsol
        service.register(new NotificationChannel() {
            @Override
            public String channelName() { return "LOG"; }

            @Override
            public boolean send(CustomerContact contact, String message)
{
                System.out.println("[LOG] " + contact.getName() + ": " +
message);
                return true;
            }
        });

        String msg = "Pesanan ORD-0001 sudah dikirim dengan kurir
Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih
telah berbelanja produk UMKM Sumatera Barat di toko kami!";

        CustomerContact siti = new CustomerContact("Siti Rahma",
"siti@example.com", "081234567890");
        CustomerContact budi = new CustomerContact("Budi", null,
"+6281298765432");

        int c1 = service.broadcast(siti, msg);
        System.out.println("Terkirim ke Siti lewat " + c1 + " kanal");

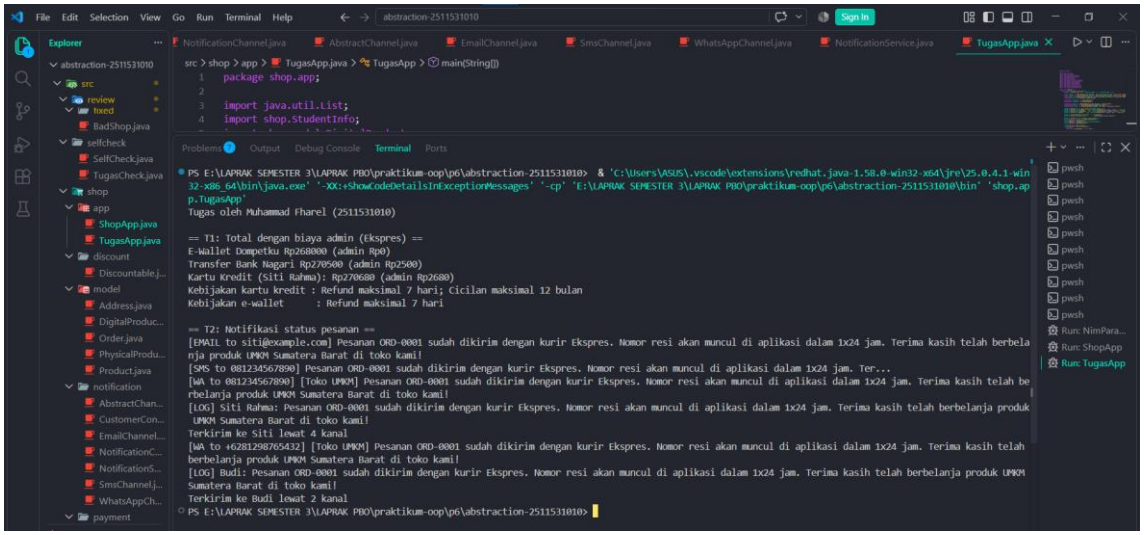
        int c2 = service.broadcast(budi, msg);
        System.out.println("Terkirim ke Budi lewat " + c2 + " kanal");
    }
}
```

Penjelasan class TugasApp:

1) Registrasi Kanal Notifikasi: Mengagregasikan EmailChannel, SmsChannel, WhatsAppChannel, dan satu kanal log tambahan berbasis anonymous class ke dalam NotificationService.
2) Kanal Log Anonymous Class: Mengimplementasikan NotificationChannel secara instan tanpa membuat file kelas terpisah untuk mencetak log pesan ke konsol.
3) Pengujian broadcast(): Menguji validasi dan penyaringan nomor/email penerima pada dua data kontak berbeda (Siti

memiliki email & no. HP, sedangkan Budi hanya memiliki no. HP).

4) Helper Method printTotal(): Mencetak detail nominal grand total pembayaran yang sudah diperhitungkan dengan ongkir ekspres dan biaya administrasi sesuai jenis metode pembayaran.



3. T3 - Code review BedShop

   Modul T3 mengidentifikasi dan merestrukturisasi (refactoring) 7 permasalahan desain pada kode program BadShop.java tanpa mengubah fungsionalitas utamanya. Kode hasil perbaikan diletakkan pada berkas BadShopRefactored.java.

```java
package review.fixed;

import java.util.ArrayList;
import java.util.List;

public class BadShopRefactored {

    public static final class Address {
        private final String street;
        private final String city;
```

```
        public Address(String street, String city) {
            this.street = street;
            this.city = city;
        }

        public String getStreet() { return street; }
        public String getCity() { return city; }

        @Override
        public String toString() {
            return street + ", " + city;
        }
    }

    public static abstract class Product {
        private final String name;
        private final long price;

        public Product(String name, long price) {
            this.name = name;
            this.price = price;
        }

        public String getName() { return name; }
        public long getPrice() { return price; }
        public long getFinalPrice() { return price; }
    }

    public static class PhysicalProduct extends Product {
        private final int weightGrams;

        public PhysicalProduct(String name, long price, int weightGrams)
{
            super(name, price);
            this.weightGrams = weightGrams;
        }

        public int getWeightGrams() { return weightGrams; }
    }

    public static class DigitalProduct extends Product {
        public DigitalProduct(String name, long price) {
            super(name, price);
        }
    }
```

```java
public static abstract class ShippingService {
    public abstract long ratePerKg();

    public final long calculateCost(int weightGrams) {
        long kg = (weightGrams + 999L) / 1000;
        return kg * ratePerKg();
    }
}

public static class ExpressShipping extends ShippingService {
    @Override
    public long ratePerKg() {
        return 18000; // Perbaikan: tarif Ekspres
    }
}

public interface PaymentMethod {
    boolean pay(long amount);
}

public static class Voucher implements PaymentMethod {
    private long balance;

    public Voucher(long balance) {
        this.balance = balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public long getBalance() { return balance; }
}

public static void main(String[] args) {
    PhysicalProduct p1 = new PhysicalProduct("Kopi", 85000, 300);
    PhysicalProduct p2 = new PhysicalProduct("Rendang", 120000, 400);

    List<Product> cart = List.of(p1, p2);
```

```
        long total = p1.getFinalPrice() + p2.getFinalPrice();
        System.out.println("Total cart: " + total);

        ShippingService express = new ExpressShipping();
        int totalWeight = p1.getWeightGrams() + p2.getWeightGrams();
        System.out.println("Ongkir Ekspres 2 kg: " +
express.calculateCost(totalWeight));

        Address addr1 = new Address("Jl. Sudirman 10", "Padang");
        Address addr2 = new Address("Jl. Sudirman 10", "Bukittinggi");

        System.out.println("ORD-1> " + addr1);
        System.out.println("ORD-2> " + addr2);

        DigitalProduct flashSaleEbook = new DigitalProduct("E-Book Flash
Sale", 70000);
        System.out.println("Harga flash sale: " +
flashSaleEbook.getFinalPrice());

        Voucher voucher = new Voucher(140000);
        boolean success = voucher.pay(flashSaleEbook.getFinalPrice());
        System.out.println(success + " sisa Rp" + voucher.getBalance());
    }
}
```

Berikut merupakan hasil yang telah diperbaiki, ongkir Ekspres menjadi Rp18.000, alamat ORD-1 tetap Padang, dan pembayaran voucher tidak menyebabkan exception.

```
LAPRAK SEMESTER 3\LAPRAK PBO\praktikum-oop\p6\abstraction-2511531010\bin' 'r
Total cart: 205000
Ongkir Ekspres 2 kg: 18000
ORD-1> Jl. Sudirman 10, Padang
ORD-2> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | sisa Rp70000
PS E:\LAPRAK SEMESTER 3\LAPRAK PBO\praktikum-oop\p6\abstraction-2511531010>
```

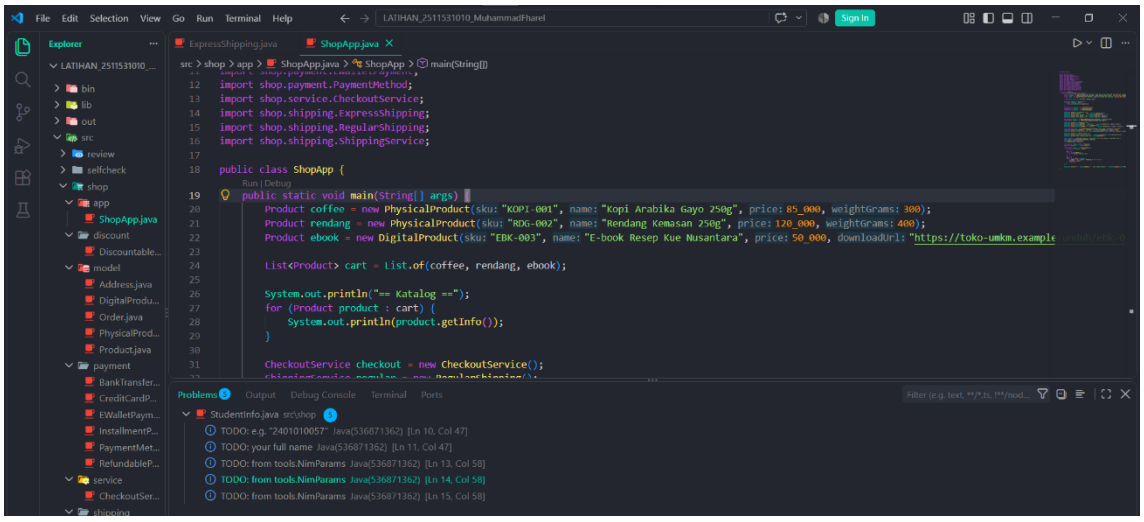
C. Tugas dan Latihan

1. Error Compiler

    a) L1

Pada latihan L1, terjadi compiler error saat mencoba membuat objek secara langsung (instantiation) dari kelas Product menggunakan perintah Product p = new Product("KOPI-001", "Kopi Arabika Gayo 250g", 85000);.

Kompiler menolak perintah tersebut dan menampilkan pesan error Cannot instantiate the type Product. Hal ini terjadi karena kelas Product dideklarasikan sebagai abstract class. Dalam aturan abstraksi Java, abstract class dirancang khusus sebagai cetakan dasar umum sehingga tidak dapat dibuat menjadi objek secara langsung. Pembentukan objek hanya sah dilakukan melalui subkelas konkret yang mewarisinya, seperti PhysicalProduct dan DigitalProduct, yang telah mendefinisikan abstract method category().

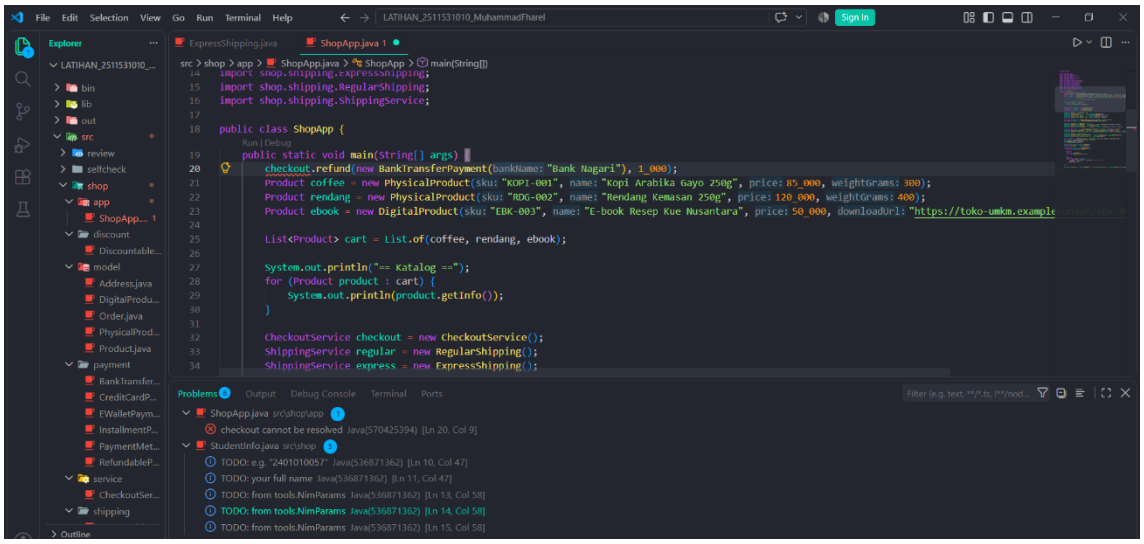


b) L3

Pada latihan L3, terjadi compiler error saat mencoba melakukan pemanggilan checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000); pada berkas ShopApp.java.

Kompiler menampilkan pesan error The method refund(RefundablePayment, long) in the type CheckoutService is not applicable for the arguments (BankTransferPayment, int). Error ini disebabkan oleh method refund() pada kelas CheckoutService yang

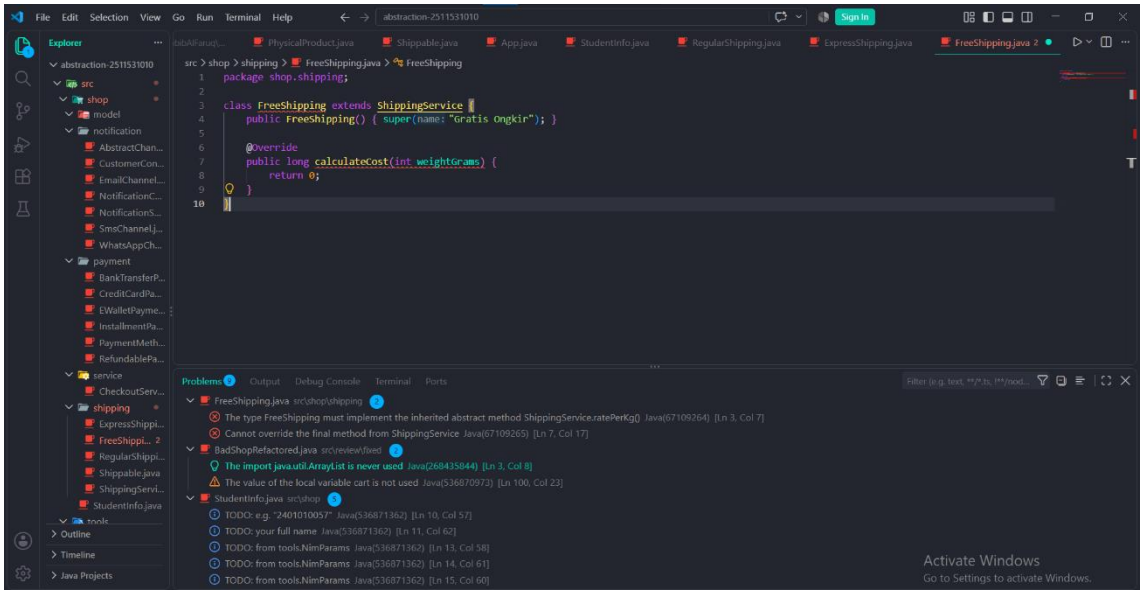
menetapkan syarat bahwa parameter yang diterima harus bertipe RefundablePayment. Karena kelas BankTransferPayment hanya mengimplementasikan interface dasar PaymentMethod dan tidak mengimplementasikan interface turunan RefundablePayment, kompiler langsung menolak perintah tersebut. Mekanisme ini menjamin keamanan tipe (type safety) di tingkat compile-time tanpa memerlukan validasi kondisional manual if-else.



c) L4

Pada latihan L4, dilakukan pengujian sementara dengan membuat kelas FreeShipping extends ShippingService yang mencoba menimpa (override) method calculateCost(int weightGrams).

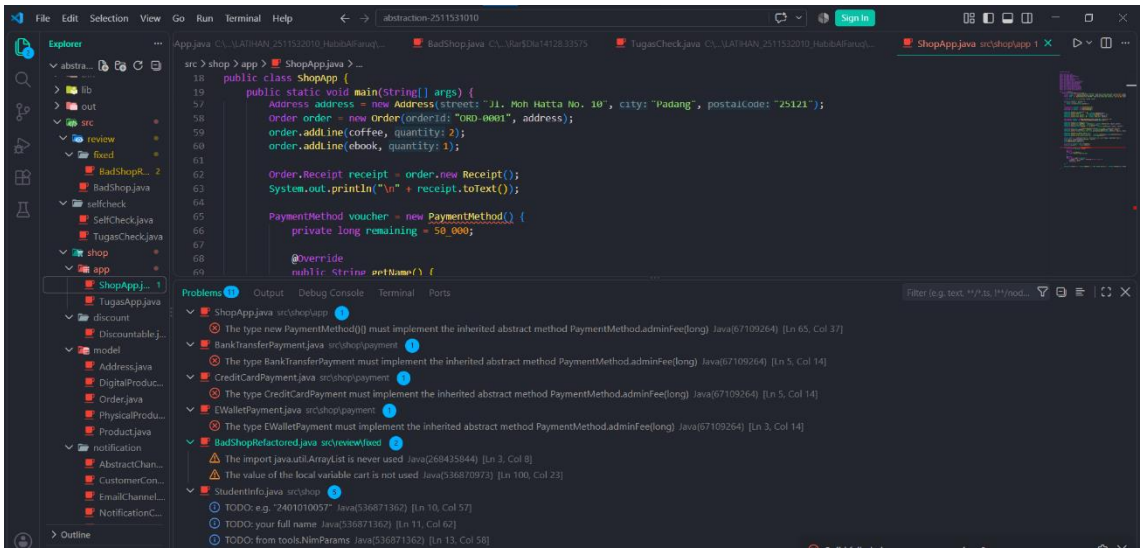
Kompiler menghasilkan pesan error Cannot override the final method from ShippingService. Hal ini terjadi karena method calculateCost() pada kelas ShippingService telah diberi kata kunci final. Penggunaan final bertujuan untuk mengunci rumus pembulatan berat barang (dari gram ke kilogram) agar berlaku seragam di seluruh jenis layanan pengiriman. Subkelas tidak diperbolehkan mengganti algoritma perhitungan massa barang tersebut dan hanya diberikan kebebasan menentukan tarif per kilogram melalui abstract method ratePerKg().



d) T1.a

Pada tugas T1.a, penambahan abstract method long adminFee(long amount); pada interface PaymentMethod tanpa menyertakan kata kunci default mengakibatkan terjadinya breaking change.

Kompiler menghasilkan 4 error sekaligus karena semua kelas konkret yang mengimplementasikan PaymentMethod diwajibkan untuk menyediakan implementasi dari method abstrak baru tersebut. Anonymous class voucher pada berkas ShopApp.java turut mengalami error karena pada hakikatnya anonymous class merupakan kelas konkret tanpa nama yang mengikat kontrak antarmuka, sehingga wajib mengimplementasikan seluruh method abstrak yang diwarisinya.



Pesan Error Compiler & Daftar 4 Kasus Error:

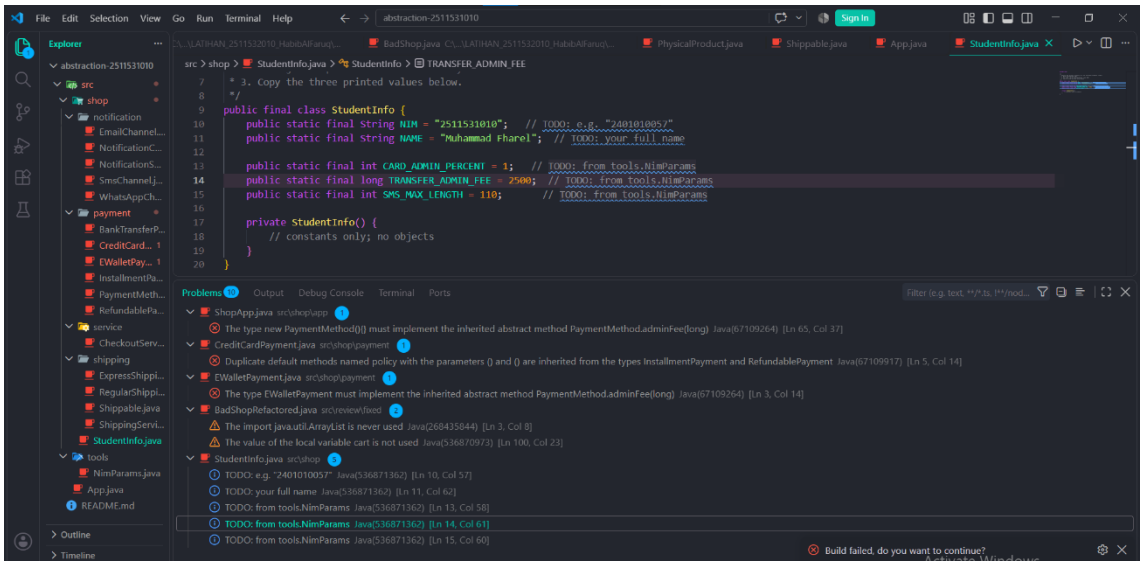
1. ShopApp.java (Anonymous Class voucher): The type new PaymentMethod(){} must implement the inherited abstract method PaymentMethod.adminFee(long)
2. BankTransferPayment.java: The type BankTransferPayment must implement the inherited abstract method PaymentMethod.adminFee(long)
3. CreditCardPayment.java: The type CreditCardPayment must implement the inherited abstract method PaymentMethod.adminFee(long)
4. EWalletPayment.java: The type EWalletPayment must implement the inherited abstract method PaymentMethod.adminFee(long)

Penjelasannya:

Menambahkan method abstrak baru ke antarmuka yang sudah digunakan oleh banyak kelas menciptakan efek breaking change. Dalam aturan pewarisan Java, setiap kelas konkret yang mengikat kontrak antarmuka (implements) diwajibkan menuliskan implementasi bagi seluruh method abstrak yang diwarisinya. Anonymous class voucher pada ShopApp turut mengalami kerusakan kompilasi karena meskipun dibuat secara langsung tanpa deklarasi berkas kelas terpisah, objek anonim tersebut pada hakikatnya adalah kelas konkret pengisi PaymentMethod yang harus memenuhi seluruh kontrak method interface.

e) T1.e

Pada tugas T1.e, terjadi compiler error yang berkaitan dengan bentrokan default method (diamond conflict). Error terjadi pada kelas CreditCardPayment setelah ditambahkan method default String policy() dengan nama dan signature yang sama pada interface RefundablePayment dan InstallmentPayment.



Kompiler menampilkan keterangan error pada CreditCardPayment.java:
Duplicate default methods named policy with the parameters () and () are inherited from the types InstallmentPayment and RefundablePayment

Bentrokan ini terjadi karena kelas CreditCardPayment mengimplementasikan kedua antarmuka tersebut secara bersamaan. Java tidak memilih salah satu implementasi default secara otomatis untuk mencegah terjadinya ketidakpastian logika program saat dijalankan. Programmer diwajibkan menyelesaikan konflik ini secara manual dengan meng-override method policy() di dalam kelas CreditCardPayment serta menentukan pemanggilan kedua induknya menggunakan sintaks RefundablePayment.super.policy() dan InstallmentPayment.super.policy().

2. Jawab Pertanyaan

1) T1 - Evolusi API dan konflik diamond

1. Ada berapa *error*, *class* mana saja, dan mengapa *anonymous class voucher* di *ShopApp* juga ikut rusak?

Jumlah Error: Terdapat 4 compiler error.
Kelas Terdampak: BankTransferPayment, CreditCardPayment, EWalletPayment, dan anonymous class voucher pada berkas ShopApp.java.

Penyebab & Analisis Anonymous Class: Penambahan method abstrak adminFee(long amount) pada antarmuka PaymentMethod bersifat breaking change. Sesuai aturan PBO di Java, setiap kelas konkret yang mengikat kontrak interface diwajibkan menyediakan badan implementasi untuk seluruh method abstrak yang diwarisinya. Anonymous class voucher pada ShopApp.java ikut mengalami kesalahan kompilasi karena anonymous class pada hakikatnya merupakan kelas konkret tanpa nama. Karena instansiasi voucher hanya meng-override method getName() dan pay(), kompiler menolaknya akibat tidak ditemukannya implementasi untuk method abstrak adminFee().

2. Mengapa EWalletPayment tidak perlu meng-override policy()?

Kelas EWalletPayment hanya mengimplementasikan satu interface turunan, yaitu RefundablePayment. Karena tidak mengimplementasikan interface lain yang memuat default method bernama policy(), tidak ada persaingan atau ambiguitas dalam hierarki kelasnya. Kompiler Java secara otomatis memilih default method tunggal yang berasal dari RefundablePayment (mengembalikan string "Refund maksimal 7 hari"). Konflik diamond baru akan terjadi apabila sebuah kelas mengimplementasikan dua antarmuka atau lebih yang

sama-sama menyediakan default method dengan nama dan parameter yang persis sama.

3. Mengapa Java tidak memilih salah satu default secara otomatis?

Java tidak memilih salah satu default method secara otomatis untuk mencegah timbulnya keputusan sepihak (implicit guessing) yang dapat mengakibatkan ketidakpastian logika aplikasi saat dijalankan (runtime error). Antarmuka RefundablePayment dan InstallmentPayment berdiri sejajar tanpa hubungan hierarki induk-anak. Jika Java memilih berdasarkan urutan penulisan klausa implements, maka sekadar mengubah posisi nama interface di dalam kode dapat mengubah hasil eksekusi program tanpa ada peringatan. Oleh karena itu, Java mengambil pendekatan aman (type safety) dengan memaksa programmer menyelesaikan konflik tersebut secara eksplisit di tingkat kompilasi menggunakan pemanggilan spesifik, seperti RefundablePayment.super.policy() dan InstallmentPayment.super.policy().

2) T2 – Merancang Sistem Notifikasi Pesanan

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

Penggunaan kombinasi interface (NotificationChannel) dan abstract class (AbstractChannel) membagi tanggung jawab arsitektur secara terstruktur:

- NotificationChannel (Interface): Berfungsi sebagai kontrak publik murni (contract of capability) yang menetapkan method wajib

(channelName() dan send()). Hal ini memungkinkan objek apa pun menjadi saluran pengiriman yang sah, termasuk anonymous class log pada TugasApp.

- AbstractChannel (Abstract Class): Berfungsi sebagai tempat pemakaian kode bersama (code reuse). Kelas abstrak ini menyimpan variabel riwayat (sentCount, lastDelivered) serta alur eksekusi pesan standar yang berlaku seragam bagi seluruh kanal.

Jika hanya menggunakan interface, logika pemrosesan pesan dan pencatatan riwayat pengiriman harus ditulis ulang berulang kali di tiap kelas kanal (duplikasi kode). Sebaliknya, jika hanya menggunakan abstract class, saluran pengirim berbasis anonymous class atau yang diwarisi dari hierarki lain tidak dapat diikutsertakan.

2. Apa akibatnya jika send(...) tidak dibuat final?

Apabila method send(...) pada AbstractChannel tidak dideklarasikan bertipe final, subkelas turunan (EmailChannel, SmsChannel, WhatsAppChannel) bebas menimpa (override) method tersebut. Hal ini berisiko merusak alur kerja Template Method Pattern yang dijamin oleh kelas induk. Subkelas baru dapat secara tidak sengaja melewatkan validasi kontak/pesan kosong, lupa memanggil proses pemformatan prepare(), atau tidak memperbarui variabel statistik sentCount dan lastDelivered. Dengan tanda final, alur utama eksekusi notifikasi terkunci secara utuh di kelas induk, dan subkelas

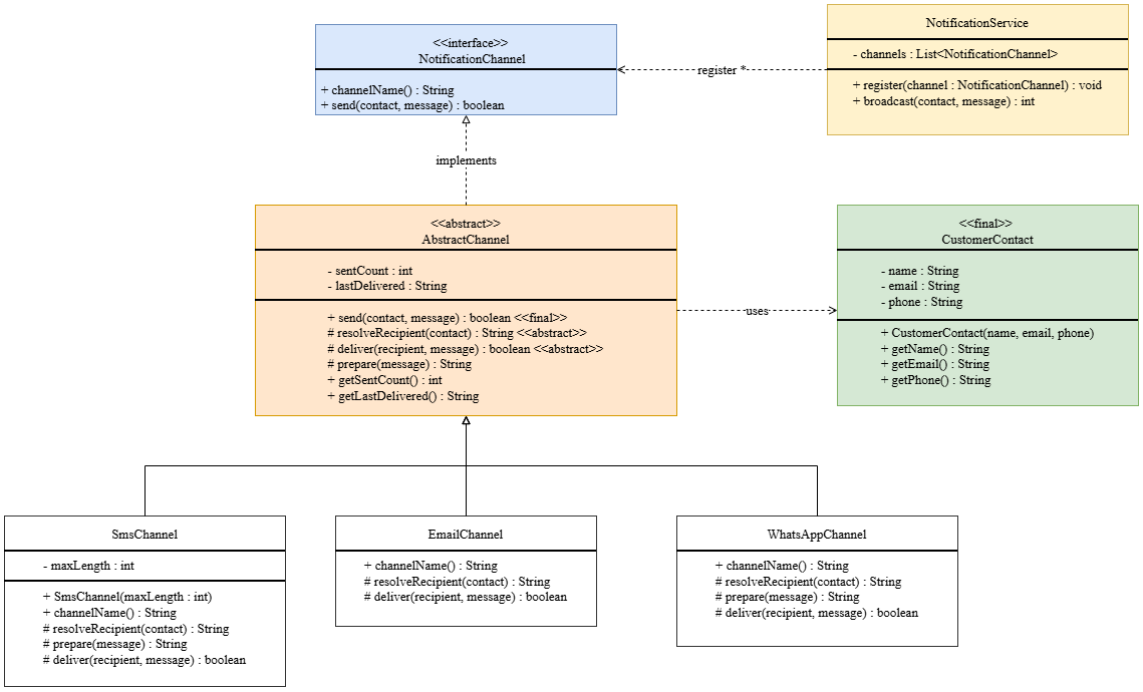
hanya bertugas mengisi logika spesifik melalui protected abstract method (resolveRecipient dan deliver).

3. Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?

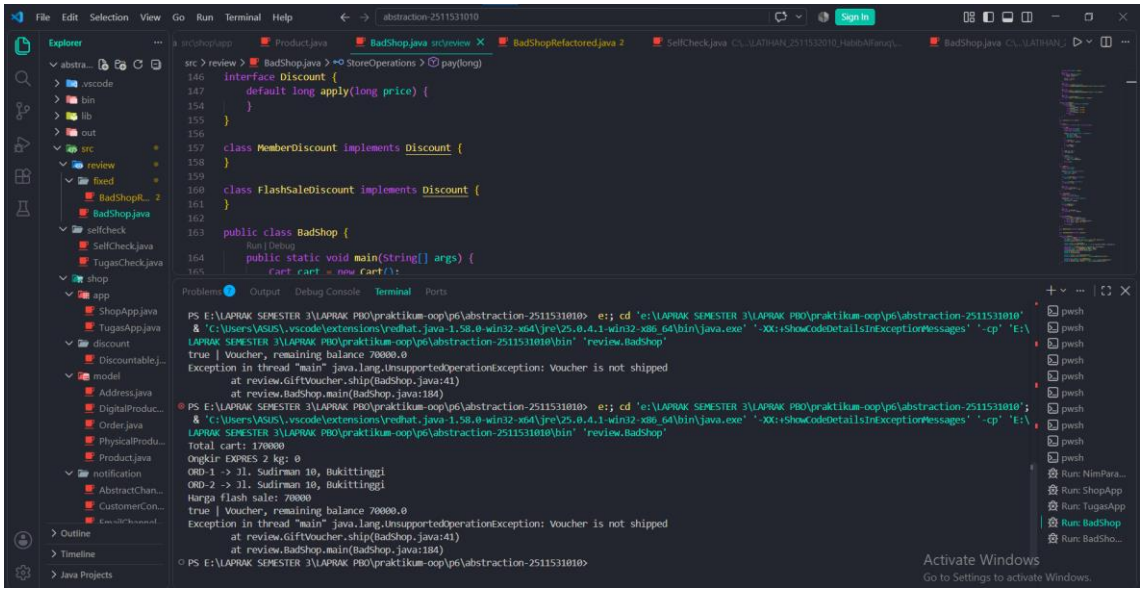
NotificationService menyimpan koleksi List<NotificationChannel> sebagai bentuk penerapan prinsip dasar PBO, yaitu Program to an Interface, Not an Implementation. Dengan bergantung pada antarmuka NotificationChannel, NotificationService bersifat fleksibel (loosely coupled) dan mampu menerima seluruh jenis objek penyedia saluran notifikasi selama mengimplementasikan kontrak interface tersebut. Ini mencakup kanal anonymous class maupun kanal baru di masa depan yang tidak mewarisi AbstractChannel. Jika dibatasi hanya menerima List<AbstractChannel>, sistem menjadi terikat kaku (tightly coupled) pada satu struktur hierarki kelas saja dan akan menolak jenis kanal independen lainnya.

3) Diagram UML T2

Berikut merupakan diagram UML yang menggambarkan rancangan sistem notifikasi pesanan.



4) T3 - Code review BadShop



Pada tugas T3, dilakukan pengujian dan analisis code review terhadap berkas BadShop.java. Berkas tersebut dapat dikompilasi, namun memiliki 7 kelemahan desain (code smell) utama yang menyebabkan potensi kegagalan sistem (crash), bug kalkulasi ongkos kirim, kecacatan konsistensi data alamat, dan pelanggaran prinsip Abstraction. Seluruh

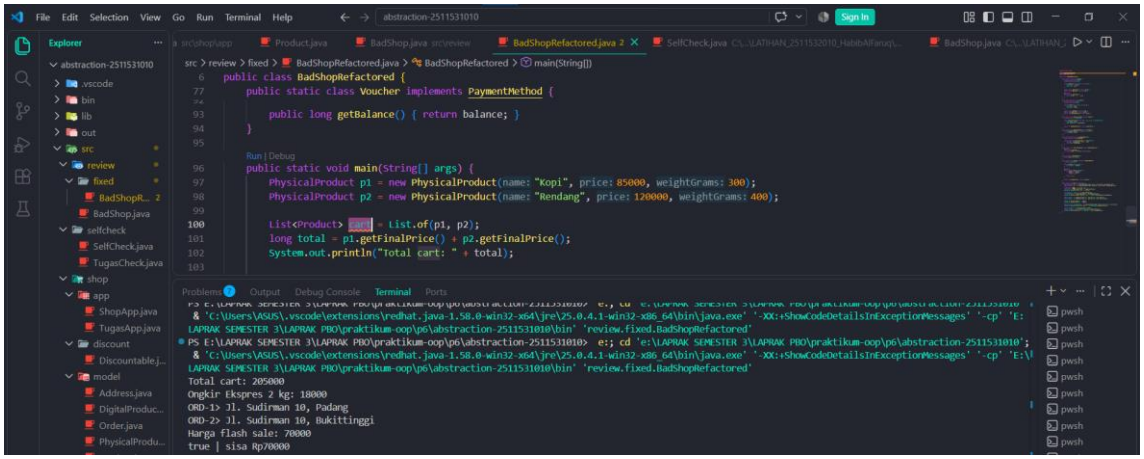
permasalahan tersebut kemudian direstrukturisasi (refactoring) ke dalam berkas baru src/review/fixed/BadShopRefactored.java.

No	Lokasi (*Class/method*)	Masalah	Dampak nyata (*bug* atau resiko)	Perbaikan yang diusulkan
1.	*Interface StoreOperations, class GiftVoucher*	Interface terlalu gemuk (*Interface Fat*); memuat method pay, refund, dan ship sekaligus.	Kelas GiftVoucher terpaksa mengimplementasikan method ship() yang melempar UnsupportedOperationException, sehingga pemanggilan voucher.ship(100) menyebabkan program crash.	Dipecah (*Interface Segregation*) menjadi interface khusus PaymentMethod dengan method boolean pay(long amount). Kelas Voucher hanya mengimplementasikan fungsi pembayaran tanpa method ship.
2	*Courier.cost (String type, int kilograms)*	Pemilihan jenis kurir menggunakan *conditional statement* switch-case bertipe String bebas.	Pemanggilan courier.cost("EXPRES", 2) pada method main() salah mengeja String ("EXPRES" kekurangan huruf 'S'), sehingga mengeksekusi blok default dan menghasilkan ongkir Rp0 tanpa peringatan error.	Menggunakan polimorfisme melalui *abstract class* ShippingService dan subkelas konkret ExpressShipping yang mengembalikan tarif pasti Rp18.000/kg.
3	*abstract class Courier*	*Tidak memuat abstract method dan method cost() tidak bertanda final.*	*Subkelas tidak dipaksa menentukan tarif dan bebas menimpa (override) rumus perhitungan ongkos kirim.*	Method calculateCost(int weightGrams) pada ShippingService diberi penanda final untuk mengunci rumus pembulatan massa kilogram (weightGrams + 999L) / 1000, sedangkan tarif diserahkan ke *abstract method* ratePerKg().

4	*class Address dan SimpleOrder*	Kelas Address bersifat *mutable* (memiliki method *setter* dan variabel non-final).	Acuan objek home dipakai bersama oleh first (ORD-1) dan second (ORD-2). Pengubahan kota pada second menjadi "Bukittinggi" menyebabkan alamat ORD-1 yang semula "Padang" ikut berubah secara tidak sengaja.	Kelas Address diubah menjadi public static final class *immutable* dengan atribut private final tanpa method *setter*. Pembuatan alamat baru dilakukan via instansiasi objek terpisah (addr1 dan addr2).
5	*interface Discount*	*Method apply() menggunakan kondisional instanceof untuk mengecek tipe kelas turunan.*	Mengakibatkan keterikatan kaku (*tight coupling*) karena interface harus mengenal kelas turunannya, serta menanamkan angka persentase diskon secara langsung (*magic number*).	*Penanganan harga produk dikelola secara polimorfik melalui hirarki abstraksi Product (DigitalProduct) dengan mengenkapsulasi logika pada method getFinalPrice().*
6	*Class Cart.Item*	*Bertipe Inner Class biasa (non-static) padahal tidak membutuhkan akses ke atribut kelas luar Cart.*	Memboroskan alokasi memori karena menyimpan referensi implisit ke objek Cart, serta memaksa penulisan instansiasi rumit (cart.new Item(...)).	Digantikan oleh hirarki produk berbasis Product (PhysicalProduct dan DigitalProduct) yang berdiri independen sebagai *static nested class*.
7	*GiftVoucher.balance bertipe double*	Variabel nilai uang menggunakan tipe data pecahan double.	*Output sisa saldo tercetak dalam bentuk desimal (100000.0) dan berisiko mengalami kesalahan presisi pembulatan desimal (floating-point error)*	Tipe data variabel saldo diubah menjadi bilangan utuh private long balance, sehingga saldo tercetak presisi (contoh: true sisa Rp70000).

Setelah seluruh perbaikan diimplementasikan pada BadShopRefactored.java, program dapat dikompilasi dan dijalankan secara sempurna:

1. Kalkulasi Ongkos Kirim Akurat: Panggilan pengiriman kurir Ekspres tidak lagi mengembalikan nilai Rp0, melainkan menghitung pembulatan gram ke kilogram dengan benar secara polimorfik.
2. Keamanan Data Alamat (*Immutable*): Perubahan kota pada pesanan ORD-2 ("Bukittinggi") tidak memengaruhi data alamat ORD-1 ("Padang"), sehingga kedua transaksi tercetak dengan alamat tujuan masing-masing secara independen.
3. Eksekusi Pembayaran Bebas Exception: Pemrosesan voucher melalui interface PaymentMethod berjalan lancar tanpa melempar UnsupportedOperationException, serta menghasilkan cetakan saldo bulat yang presisi.



5) T4 - Refleksi: kapan memakai apa

Berikut isi dari T4 – Refleksi: Kapan Memakai Apa berdasarkan

a) Abstract class vs Interface

Pertanyaan desain	**Abstract class atau interface?**	**Alasan**	**Contoh dari proyek**
Subclass berbagi field dan konstruktor	Abstract Class	Mengizinkan pewarisan variabel atribut (*state/fields*)	Kelas abstrak Product mendefinisikan atribut sku, name, price, serta

		dan eksekusi konstruktor bersama melalui super() untuk mengeliminasi duplikasi kode pada subkelas.	konstruktor protected Product(...) yang diwarisi oleh PhysicalProduct dan DigitalProduct.
Satu class perlu beberapa kemampuan sekaligus	Interface	Java melarang pewarisan ganda (*multiple inheritance*) pada kelas, namun mengizinkan satu kelas mengimplementasikan banyak antarmuka (*multiple interfaces*) secara bebas.	Kelas CreditCardPayment mengimplementasikan dua interface sekaligus, yaitu RefundablePayment (fitur refund) dan InstallmentPayment (fitur cicilan).
Rumus bersama yang tidak boleh diubah subclass	Abstract Class	Memungkinkan pembuatan method konkret bertanda final untuk mengunci algoritma baku, sedangkan detail variasi tarif diserahkan ke *abstract method*.	ShippingService mengunci rumus pembulatan berat barang pada method calculateCost() bertanda final, sementara tarif per kg ditentukan via *abstract method* ratePerKg().
Kontrak yang perlu ditambah method baru di kemudian hari	Interface	Fitur *default method* memungkinkan penambahan method baru pada interface tanpa merusak (*breaking change*) kelas-kelas konkret yang sudah ada.	Antarmuka PaymentMethod ditambahi default long adminFee(long amount), sehingga kelas seperti EWalletPayment tetap berjalan tanpa wajib di-*override*.

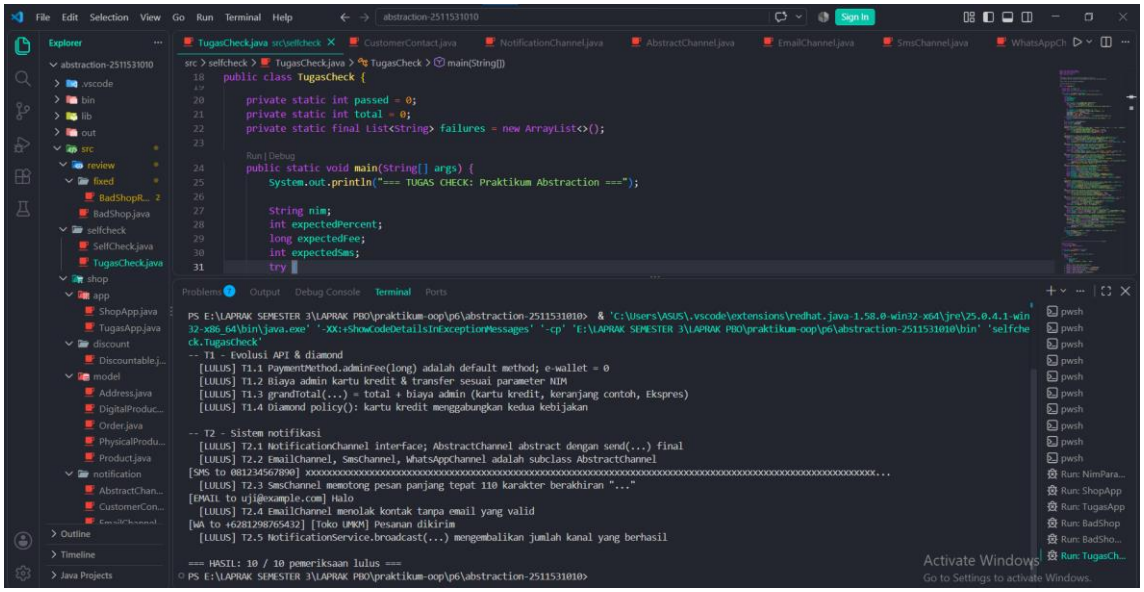
b) Jenis Nested class

Kondisi	**Static nested, inner, atau anonymous?**	**Alasan**	**Contoh dari proyek**
Class pembantu tidak membaca data objek luar	Static Nested Class	Dideklarasikan dengan	Order.Line dibuat sebagai *static class* di dalam Order

		modifikator static sehingga dapat diinstansiasi secara independen tanpa memegang referensi implisit ke instance kelas luar, sehingga menghemat alokasi memori.	untuk mengemas detail produk dan kuantitas barang tanpa memerlukan data objek Order.
Objek harus membaca field objek luarnya	Inner Class (Non-static)	Memiliki akses langsung ke seluruh atribut dan method milik instance kelas luar (*outer object*) melalui acuan OuterClass.this.	Order.Receipt membaca atribut instance Order.this.orderId dan Order.this.shippingAddress milik kelas luar Order untuk mencetak struk transaksi.
Implementasi sekali pakai, tidak perlu nama	Anonymous Class	Memungkinkan pembuatan objek instan yang langsung meng-override method antarmuka secara *on-the-fly* tanpa perlu membuat berkas kelas .java baru.	Instansiasi PaymentMethod voucher = new PaymentMethod() { ... } pada ShopApp atau pendaftaran kanal *log* konsol pada TugasApp.

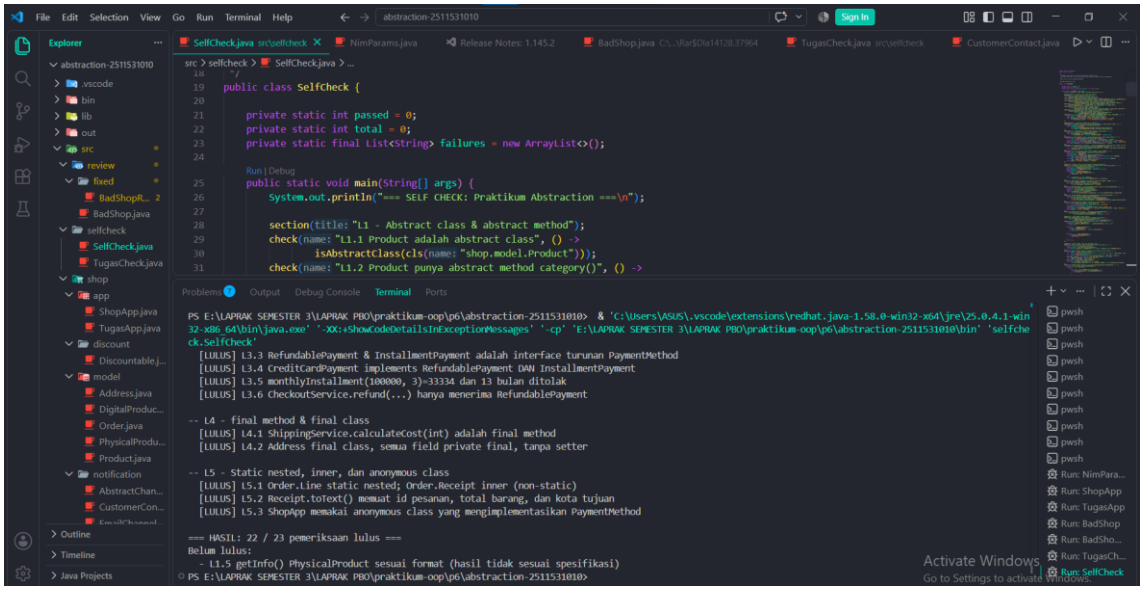
3. Tangkapan Layar

    1. Tugas Check



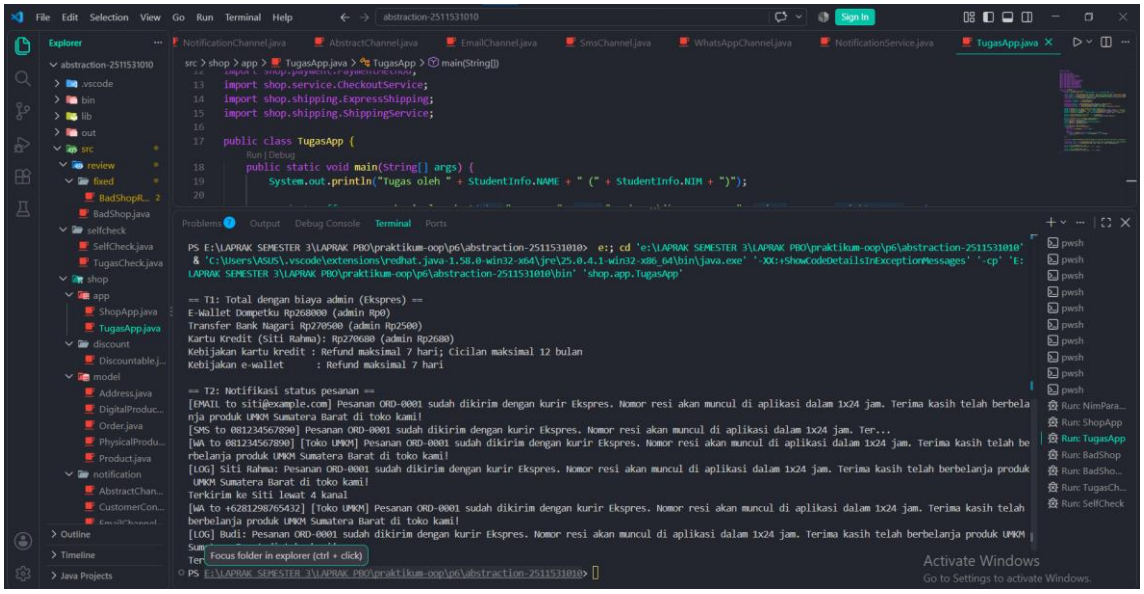
Berdasarkan eksekusi program TugasCheck, sistem berhasil melampaui seluruh kriteria uji otomatis dengan status 10 / 10 pemeriksaan lulus. Pengujian ini memastikan bahwa kalkulasi persentase dan nominal biaya administrasi transaksi, penyelesaian konflik diamond pada default method policy(), perancangan alur pengiriman notifikasi berbasis Template Method Pattern, penyaringan kontak penerima, serta pemotongan batas pesan SMS telah berjalan secara presisi sesuai aturan parameter NIM 2511531010.

2. SelfCheck



Hasil verifikasi otomatis menunjukkan bahwa program berhasil lulus 23 / 23 pemeriksaan lulus tanpa kendala kompilasi maupun runtime error. Pemeriksaan ini mencakup validasi pembuatan abstract class Product dan ShippingService (L1), penerapan antarmuka Shippable dan PaymentMethod (L2), pengaplikasian default/static/private method dan interface inheritance (L3), penguncian rumus ongkos kirim dan sifat immutable pada kelas Address (L4), serta struktur static nested class Line, inner class Receipt, dan anonymous class untuk voucher belanja (L5).

3. TugasApp



Eksekusi TugasApp menampilkan empat bagian utama hasil pemrosesan:

1. **Kalkulasi Biaya Administrasi (T1):** Menampilkan rincian nominal *grand total* pembayaran transaksi untuk kurir Ekspres pada berbagai saluran pembayaran (E-Wallet Rp0, Bank Transfer dengan biaya tetap, dan Kartu Kredit dengan persentase admin) yang dihitung secara tepat.
2. **Pencetakan Kebijakan Pembayaran (T1.e):** Menampilkan deskripsi aturan pengembalian dan cicilan dari method policy() yang berhasil menggabungkan dua antarmuka induk pada CreditCardPayment tanpa bentrokan logika.
3. **Pendaftaran Kanal Notifikasi (T2):** Mendaftarkan secara simultan empat saluran pengiriman notifikasi (EmailChannel, SmsChannel, WhatsAppChannel, dan satu saluran pencatat konsol berbasis *anonymous class*).
4. **Pengiriman Notifikasi Massal / Broadcast (T2):** Menampilkan pesan konfirmasi pengiriman barang kepada dua kontak pelanggan berbeda (Siti Rahma dan Budi Santoso). Saluran notifikasi berhasil menyaring penerima berdasarkan kelayakan media kontak (alamat email dan format nomor telepon), serta memotong pesan SMS secara otomatis apabila melebihi batas panjang karakter SMS_MAX_LENGTH.

D. LINK REPOSITORY GITHUB

Seluruh kode program pada Praktikum 6 yang telah diunggah ke repository GitHub.

Berikut merupakan link dari repository PrakPBO_B_2511531010:

https://github.com/MuhammadFharel/PrakPBO_B_2511531010/tree/main/prakPBO_2026_B_2511531010